The Knowledge–Action Chain (KAC): An AX Execution Ontology for Turning Validated Knowledge into Action, Outcome, and Learning

From knowledge flow to verified execution — a relational model linking skill derivation, runtime, action events, outcomes, review, and context feedback

Ryo Ha · Sang Hoon Park · Woo Bin Jeong

sopia19910@gmail.com · pak3430@gmail.com · wjdnqs7@gmail.com

June 23, 2026

ABSTRACT

In the age of AI, the core problem of organizational operation is no longer "can AI produce an answer?" Generative and agentic AI already participate deeply in drafting documents, retrieving information, analysis, decision support, tool execution, and artifact generation — yet an AI-produced answer does not automatically become an organizational execution capability. The real question is whether validated knowledge is derived into a specific **Skill** within a given domain, whether that Skill is compiled into an executable **Runtime**, whether it produces an actual **ActionEvent** with distinct **OutputObject** and **OutcomeObject**, and whether it passes **Review** and **Feedback** to update the organization's domain context. This paper defines that entire structure as the **Knowledge–Action Chain (KAC)**. KAC connects the external knowledge chain, the skill derivation chain, SkillRuntime, ActionEvent, OutputObject, OutcomeObject, Review, Feedback, and Context Update into a single execution ontology. The *Identity* → *Goal* → *Task* → *Knowledge* → *Method* → *Skill* structure previously called the "knowledge chain"**[R1]** is, strictly speaking, not the whole knowledge chain but a **skill derivation chain** explaining why a given Identity or Entity requires a given Skill; KAC closes the execution, verification, and learning structure that follows. The central thesis is simple — **a knowledge chain is a path of knowledge; a knowledge–action chain is the path along which knowledge is verified through action.** We present the nine-tuple definition of KAC, the prior gate DCI**[R6]**, the execution contract of SkillRuntime, the separation of ActionEvent / OutputObject / OutcomeObject, the validity condition ValidKAC, and the operational linkage with DCI, KCDI, RCI, AHCI, and ARBI.

Keywords Knowledge–Action Chain, KAC, skill derivation chain, SkillRuntime, ActionEvent, OutcomeObject, domain context, DCI, KCDI, RCI, AHCI, ARBI, organizational AX

1 Problem: Why Knowledge Must Close into Action

Organizations in the age of AI are no longer at the stage of deciding "whether to use AI." Many already use AI to draft documents, summarize meetings, write customer-response drafts, analyze strategy materials, search internal knowledge, and chain multiple tools to execute work. But an increase in AI usage does not complete organizational AX. The essence of AX is not using AI a lot, but **making AI operate repeatably within the organization's structure of purpose, criteria, roles, authority, verification, records, accountability, and improvement.**

The common context is the first answer to this problem. A common context is the shared operating standard through which humans and AI think and work by the same criteria, composed of purpose, meaning, situation, criteria, role, limit, source, format, and feedback**[R4]**. But a common context alone is not enough. An organization has many domains — customer service, legal, education, marketing, development, executive reporting, ESG — and each has its own language, knowledge, workflow, risk criteria, verification criteria, and approval structure. The common context must therefore be instantiated into a per-domain execution context, and that domain context must be verified for use as a genuine reference environment — this is the role of DCI (Domain Context Integrity)**[R6]**. The governance context, in turn, is the network operating order determining in what context AI judges, by what authority it executes, by what criteria it is verified, who approves it, where it is recorded, and how accountability is held**[R4]**. The core question now shifts:

> How is validated knowledge converted into actual action?
> How does a derived Skill become an executable Runtime?
> As what ActionEvent is the execution recorded?
> As what OutputObject does the artifact remain, and as what OutcomeObject is the change confirmed?
> Which Review does the result pass, and which Feedback enters the next version of the domain context?

The structure that answers these questions is precisely the **Knowledge–Action Chain (KAC)**.

2 Terminology: Knowledge Chain, Skill Derivation Chain, Knowledge–Action Chain

To understand KAC precisely, three structures must first be separated (*Table 1*). First, the **external knowledge chain (Knowledge Chain_ext)** is the flow through which knowledge is created, connected, inferred, accumulated, transmitted, applied, and reused. The Knowledge Chain, Knowledge Value Chain, and Knowledge Management Chain in the external literature mainly address the acquisition, creation, storage, sharing, application, and reuse of knowledge**[R7],[R8]**. Second, the **skill derivation chain (SDC)** is the reasoning path *Identity* → *Goal* → *Task* → *Knowledge* → *Method* → *Skill* that derives why a given Identity or Entity requires a given Skill**[R1]**. Third, the **knowledge–action chain (KAC)** is the AX execution chain that, building on the two chains, further includes SkillRuntime, ActionEvent, OutputObject, OutcomeObject, Review, Feedback, and Context Update.

Table 1. Core question and role of the three structures

Structure	Core question	Role
External knowledge chain	Where does knowledge come from, and how is it connected, stored, shared, applied?	Knowledge flow
Skill derivation chain	Why does a given Identity/Entity require a given Skill?	Knowledge → skill requirement
Knowledge–action chain	**Did the derived Skill come alive as Runtime, Action, Outcome?**	**AX execution, verification, learning**

Why this distinction matters is clear. *Identity* →… → *Skill* is a crucial structure, but by itself it is not yet action — it is the chain explaining why a Skill requirement is justified. KAC covers everything up to the moment that Skill actually enters the operational world.

3 Core Definition of KAC

KAC is defined as follows — **the Knowledge–Action Chain is the AX execution–verification–learning chain along which validated knowledge is derived into a Skill within a specific domain context, transformed into an executable state through a SkillRuntime, and passed through actual ActionEvent, OutputObject, OutcomeObject, Review, and Feedback to update the domain context.** More briefly, it is "the chain that converts knowledge into executable organizational capability," and most compactly, "the path along which knowledge is verified through action." The basic path is shown in *Figure 1*.

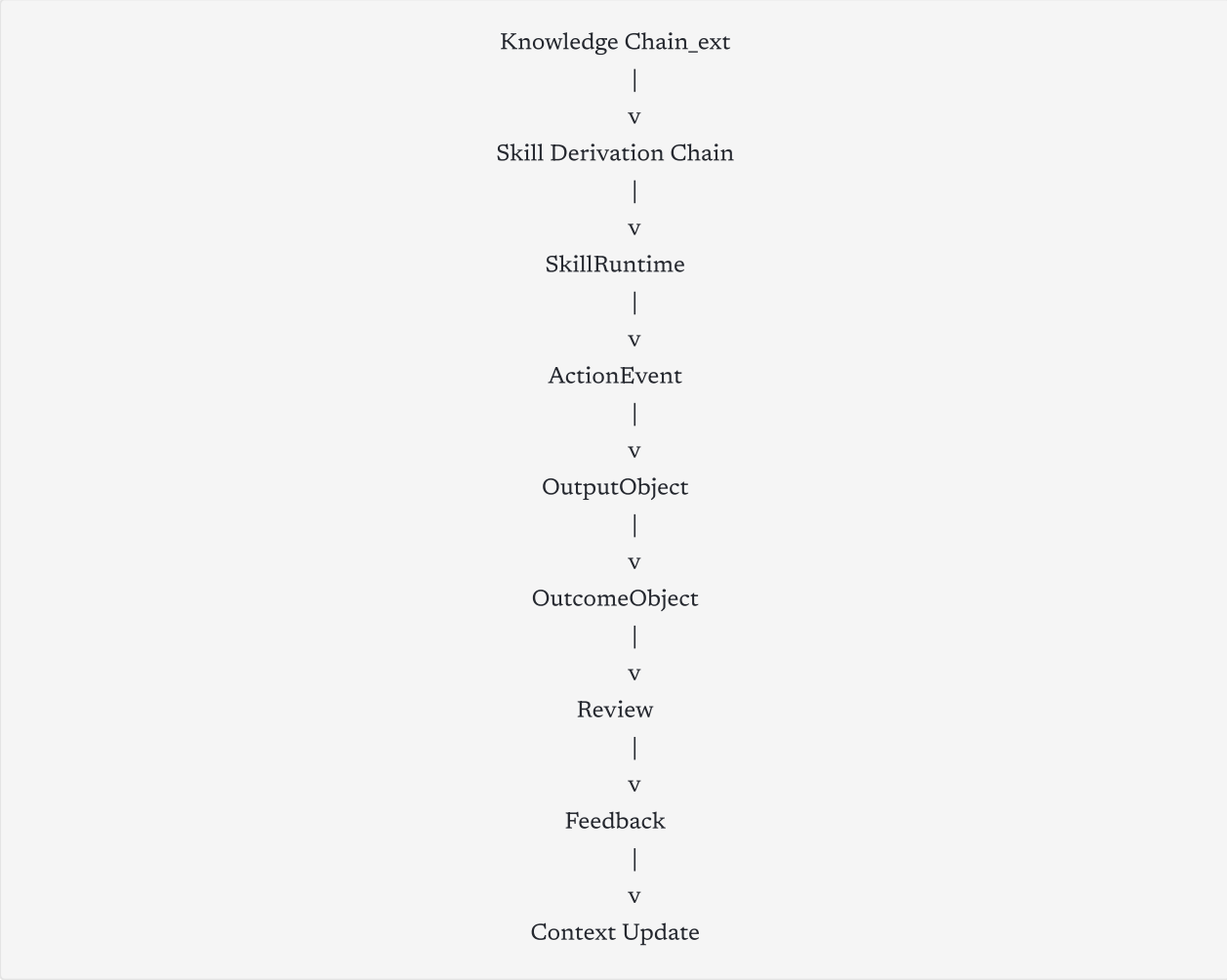


Figure 1. The basic path of KAC — starting from the external knowledge chain, it passes through skill derivation, the execution contract, the action event, the artifact, the outcome, review, and feedback, closing with the update of the domain context.

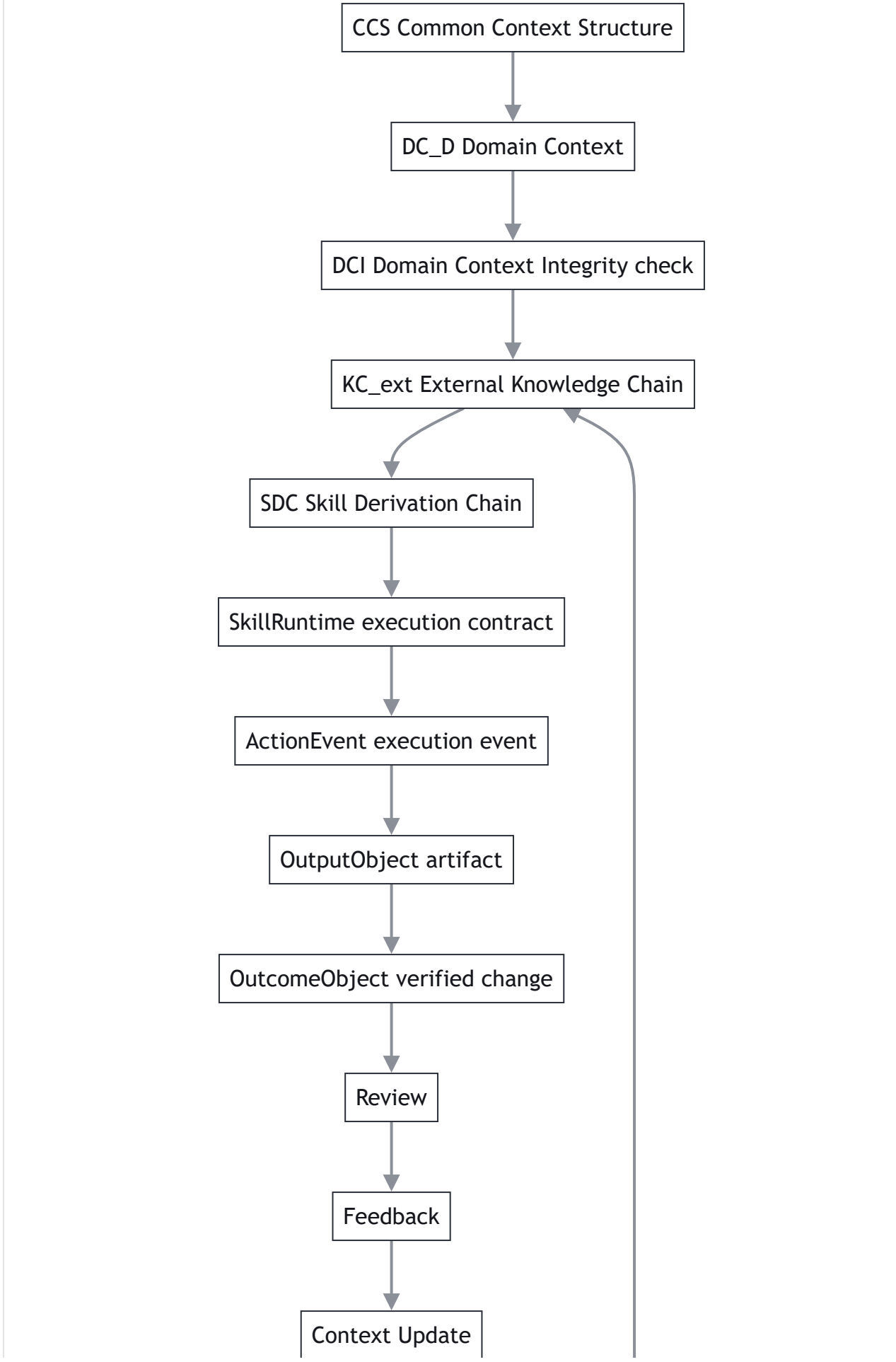
As an object-form equation, it is a nine-tuple (*Table 2*).

$$KAC = \langle KC^{ext}, \mathrm{SDC}, R, A, P, \Omega, V, F, DC^{D}(t+1) \rangle \quad (1)$$

Table 2. The nine components of KAC

Symbol	Name	*Symbol*	Name / meaning
KC_ext	External Knowledge Chain	Ω	OutcomeObject (verified change)
SDC	Skill Derivation Chain	V	Review
R	SkillRuntime (execution contract)	F	Feedback
A	ActionEvent (execution event)	$DC\_D(t+1)$	Updated domain context
P	OutputObject (artifact)		

This nine-tuple treats KAC not as a conceptual description but as a connection of operational objects. That is, KAC asks not "did AI produce an answer?" but "which knowledge was derived into which Skill, executed through which Runtime, produced which ActionEvent, artifacts, and outcomes, and reflected into organizational criteria through which review and feedback?"


CCS Common Context Structure
DC_D Domain Context
DCI Domain Context Integrity check
KC_ext External Knowledge Chain
SDC Skill Derivation Chain
SkillRuntime execution contract
ActionEvent execution event
OutputObject artifact
OutcomeObject verified change
Review
Feedback
Context Update

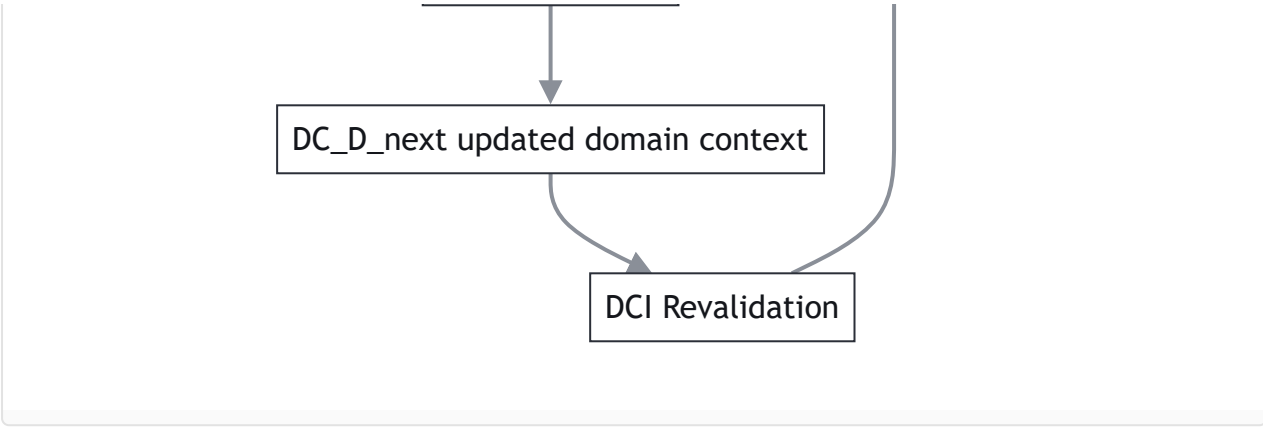


Figure 2. KAC basic cycle — from the common context structure through the domain context and DCI, then the external knowledge chain, skill derivation, execution contract, action event, output, outcome, review, feedback, and context update, closing back into DCI revalidation.

4 Prerequisite: The Domain Context and DCI

KAC is not a free-floating execution chain. For it to hold, a specific domain context must first stand as a reference environment. The common context structure (CCS) is the common operating grammar every domain context must follow, and the domain context (DC_D) is the execution world that applies that grammar to a specific domain D**[R6]**.

$$\text{CCS} = \langle \text{Purpose, Meaning, Situation, Criteria, Role, Limit, Source, Format, Feedback} \rangle \tag{2}$$

$$DC^{D} = \text{Instantiate}(\text{CCS}, D) = \langle D, \text{CCS}, I^{D}, G^{D}, T^{D}, K^{D}, M^{D}, S^{D}, E^{D}, \text{Src}^{D}, \text{Val}^{D}, \text{Gov}^{D}, \text{Fb}^{D} \rangle \tag{3}$$

DCI verifies whether this domain context is usable as a reference. The I in DCI stands for **Integrity**, not Index — it is a consistency verification, not a performance score: "may this domain context be used as a reference?" The full formula is the product of a blocking gate, a completion gate, quality, and risk attenuation.

$$DCI^{V}(D) = G^{B}(D) \cdot G^{C}(D) \cdot Q^{DCI}(D) \cdot \text{RA}(D) \tag{4}$$

The key is that gates come before scores. Fatal defects — absent domain owner, absent operating authority, ingested sensitive data, unspecified final human accountability, source violations, absent minimum governance — cannot be compensated by a high quality score. So the entry condition of KAC is clear — **ValidKAC begins only after the domain context passes DCI.** If the domain context does not stand as a reference environment, the knowledge chain, SkillRuntime, ActionEvent, Review, and Feedback built upon it all become unstable.

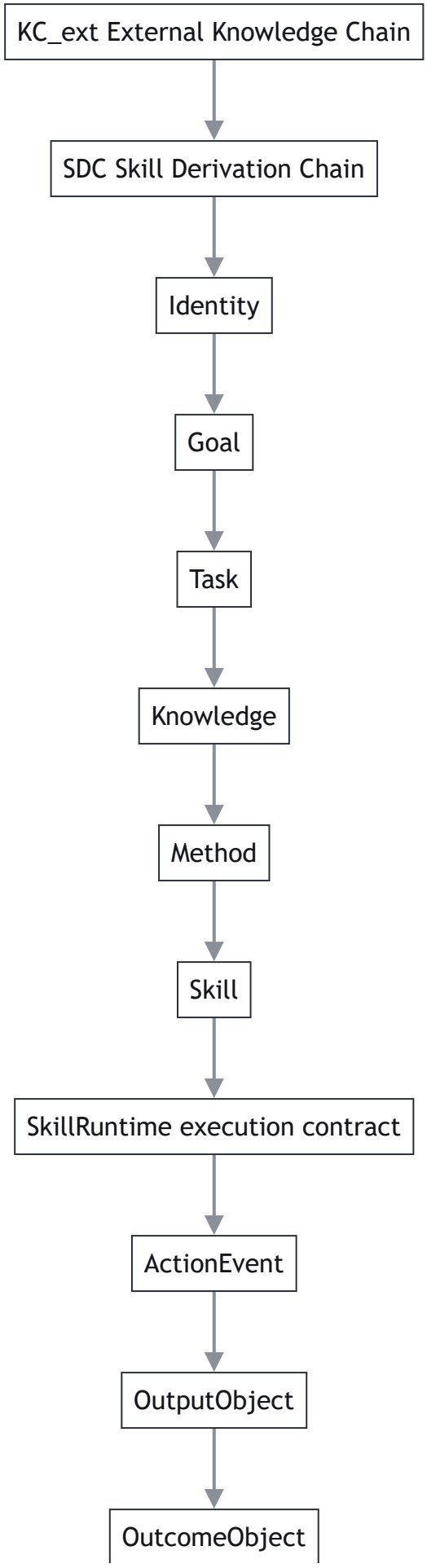
5 The Skill Derivation Chain: KAC's First Half

KAC's first half is the skill derivation chain *Identity* → *Goal* → *Task* → *Knowledge* → *Method* → *Skill*. What matters here is the separation of Identity and Entity. An Identity is an existential definition, role, or meaning structure; an Entity is the actual agent that realizes that Identity. For example, *OxfordApplicant* is an Identity, and an actual applicant such as *Sloane* is an Entity. An Identity has an intension (the meaning structure of the Goals, Tasks, Knowledge, Methods, and Skills it requires) and an extension (the set of Entities that realize it)**[R1]**. This separation is decisive in KAC – KAC asks not only "what must this person do?" but "what Skill does the Entity realizing this Identity require, and through which Runtime and Action was that Skill executed?"

The skill derivation chain decomposes at the Knowledge node into development and deployment.

$$\begin{aligned} \pi^{dev} &= \text{Identity} \to \text{Goal} \to \text{Task} \to \text{Knowledge}, \quad \pi^{dep} \\ &= \text{Knowledge} \to \text{Method} \to \text{Skill}, \quad \pi = \pi^{dev} \oplus_K \pi^{dep} \end{aligned} \tag{5}$$

The development segment fixes the meaning of an identity as a knowledge requirement; the deployment segment converts that knowledge into method and skill. Knowledge holds the dual status of being the output of development and the input of deployment**[R1]**. But the creation of a Skill node does not complete KAC – a Skill is not yet execution but a derived capability requirement, and for that requirement to lead to actual action it needs an executable contract structure. This is the SkillRuntime.


KC_ext External Knowledge Chain
SDC Skill Derivation Chain
Identity
Goal
Task
Knowledge
Method
Skill
SkillRuntime execution contract
ActionEvent
OutputObject
OutcomeObject

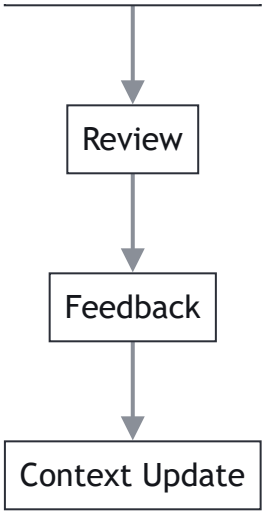


Figure 3. Inside the SDC and KAC latter half — the skill derivation chain (Identity→Skill) unfolds from the external knowledge chain, and the derived Skill continues into SkillRuntime, ActionEvent, output, outcome, review, feedback, and context update.

6 SkillRuntime: The Pivotal Transition in KAC

The most important transition in KAC is *Skill* → *SkillRuntime*. A Skill is not execution but an execution requirement. Deriving a "policy-grounded customer-service skill" does not immediately execute customer service; inputs, outputs, sources, tools, authority, verification, records, approval, state management, and release policy must be specified. Thus — **a SkillRuntime is the structure that compiles a derived Skill into an execution contract from which an actual ActionEvent can occur.** It is not a single prompt or a single file but a contract graph containing multiple execution conditions (*Table 3*).

$$\begin{aligned}\text{SkillRuntime} = {} & \text{SkillInterface} + \text{InputContract} + \text{OutputContract} + \text{SourcePolicy} + \\ & \text{ToolPolicy} + \text{Guardrail} + \text{ValidationRule} + \text{TracePolicy} + \text{ApprovalPolicy} + \\ & \text{WorkflowSpec} + \text{InvocationContract} + \text{StateModel} + \text{ReleasePolicy}\end{aligned} \tag{6}$$

Table 3. Components and roles of SkillRuntime

Component	Role	*Component*	Role
SkillInterface	External touchpoint	*TracePolicy*	Records grounds and logs
InputContract	Input schema	*ApprovalPolicy*	Human approval conditions
OutputContract	Output schema	*WorkflowSpec*	Invocation order and conditions
SourcePolicy	Usable sources	*InvocationContract*	Invocation contract
ToolPolicy	Callable tools	*StateModel*	Execution state management
Guardrail / ValidationRule	Risk and quality control	*ReleasePolicy*	Release and version policy

A Skill without a SkillRuntime remains an unexecuted capability requirement. Conversely, with a SkillRuntime a Skill becomes an executable operational object. This is what makes KAC an AX execution ontology rather than a mere theory of knowledge.

7 ActionEvent, OutputObject, OutcomeObject

When a SkillRuntime is invoked, an actual execution event, the **ActionEvent**, occurs. An ActionEvent is the moment knowledge enters the operational world; it must be not a mere "task performed" but an execution record of who, when, which Runtime, with what input, and under what authority, source, and tool policy invoked it.

$$\text{ActionEvent} = \langle\, \text{actor, runtime, input, source, tool, time, permission, trace, status} \,\rangle \tag{7}$$

As a result of the ActionEvent, an **OutputObject** (artifact) is produced — a customer-response draft, a legal review memo, a report, a summary, code, a workflow execution result, an evaluation sheet. But an OutputObject and an **OutcomeObject** differ. An OutputObject is the artifact produced; an OutcomeObject is the actual change that artifact caused.

$$\text{OutputObject} = \text{produced artifact}, \quad \text{OutcomeObject} = \text{verified change caused or supported by the output} \tag{8}$$

For instance, if AI drafts a customer reply, that draft is an OutputObject; if the reply resolves the inquiry, reduces re-inquiries, involves no policy violation, and raises satisfaction, that is the OutcomeObject. This distinction is decisive in KAC — many AI deployments stop at artifact generation, but KAC asks not "what did AI produce?" but "what change did it produce, and was that change verified?"

8 Review, Feedback, and Context Update

KAC does not end at the OutcomeObject. An OutcomeObject must pass **Review**, and this review is not a mere quality assessment but includes authority, accountability, source, grounds, criteria, expression, approval, records, and risk. RCI concretizes this review at the utterance and artifact level: it treats an AI-augmented utterance as a reviewable operational object, handling authority and accountability violations through a blocking gate and unmet object, evidence, and record conditions through a revision gate, and evaluating the quality of passing candidates through a weighted geometric mean and a risk attenuation term**[R3]**. A result that passes review is organized into **Feedback** — not a mere opinion but a context change candidate.

$$\text{Feedback} = \langle \text{issue, evidence, affected\_context, proposed\_change, risk, owner, approval\_status, rollback\_plan} \rangle \tag{9}$$

When feedback is approved, a new version of the domain context is created, and the new domain context must pass DCI again.

$$DC^{D}(t) + \text{ApprovedFeedback} \rightarrow DC^{D}(t+1) \rightarrow \text{DCI Revalidation} \tag{10}$$

This structure makes KAC not a one-off execution but an organizational learning chain — it closes from knowledge to action, from action to outcome, from outcome to review, from review to feedback, and from feedback to context update.

9 ValidKAC: The Validity Condition of KAC

KAC does not hold automatically because a path exists; each segment must be verified (*Table 4*).

$$\text{ValidKAC} = \text{ValidKC} \wedge \text{ValidSDC} \wedge \text{RuntimeReady} \wedge \text{ValidAction} \wedge \text{ValidOutput} \wedge \text{ValidOutcome} \wedge \text{ReviewPassed} \wedge \text{FeedbackValid} \wedge \text{ContextRevalidated} \tag{11}$$

Table 4. The nine validity conditions of ValidKAC

Condition	Verification question
ValidKC	Was the external knowledge chain built along permitted knowledge relations and sources?
ValidSDC	Was the Skill justifiably derived from an Identity or Entity basis?
RuntimeReady	Was the Skill compiled into an executable Runtime contract?
ValidAction	Was the ActionEvent performed within authority, source, tool, and guardrail limits?
ValidOutput	Does the OutputObject satisfy the schema and quality criteria?
ValidOutcome	Does the OutcomeObject show an actual change against the goal criteria?
ReviewPassed	Did the result pass a review confirming it is reflectable?
FeedbackValid	Does the feedback carry grounds, scope of effect, owner, approval, and a rollback plan?
ContextRevalidated	Did the updated domain context pass DCI again?

The meaning of this equation is strong – even if the knowledge is good, if the Skill is not justifiably derived; even if the Skill is justified, if there is no Runtime; even if an Action occurs, if the Output falls short; even if there is Output, if there is no Outcome; even if there is Outcome, if it fails Review; even if there is Feedback, if it fails DCI revalidation – KAC does not hold. KAC is therefore not the loose claim that "knowledge was executed," but is completed only when knowledge, skill, execution, output, outcome, review, feedback, and context update all hold.

10 Linking KAC with Operational Metrics

KAC becomes an actual operating system when linked with DCI, KCDI, RCI, AHCI, and ARBI (*Table 5*). **DCI** is the prior gate verifying whether a domain context is usable as a reference environment; it does not guarantee KCDI but provides the reference environment in which KCDI can be meaningfully measured**[R6]**. **KCDI** measures the deployment success of the knowledge chain – "did the derived Skill come alive as an actual execution capability?" – and is a product of six factors**[R2]**.

$$\mathrm{KCDI}(e,\pi,c)=\mathrm{ValidDeployment}(\pi)\cdot\mathrm{Adopt}(e,m,c)\cdot\mathrm{Use}(e,m,c)\cdot\mathrm{Improve}(e,s)\cdot\mathrm{Trace}(\pi,e,c)\cdot\mathrm{GovernanceFit}(c) \tag{12}$$

The multiplicative structure guarantees that a single fatal defect collapses the whole success – even a logically well-derived Skill, if unused in the field, or used but without records or in violation of governance, cannot be regarded as a deployment success. **RCI** verifies KAC's later-stage Review at the level of utterances, claims, responses, and artifacts**[R3]**; **AHCI** evaluates whether, in a 1:1 human–bot collaboration, the human maintains and expands the capabilities of goal setting,

context provision, execution direction, result interpretation, verification, accountability, recording, and improvement; and **ARBI** evaluates whether, in AI-mediated collaboration, the roles of humans and AI are healthily divided, complemented, verified, and held accountable**[R5]**.

Table 5. The five metrics of the KAC operating system

Metric	Role
DCI	Verifies the reference environment (domain context integrity)
KCDI	Measures field operation of the Skill (deployment success)
RCI	Verifies reflectability of artifacts and utterances
AHCI	Measures human–AI augmentation capability
ARBI	Measures collaborative role balance

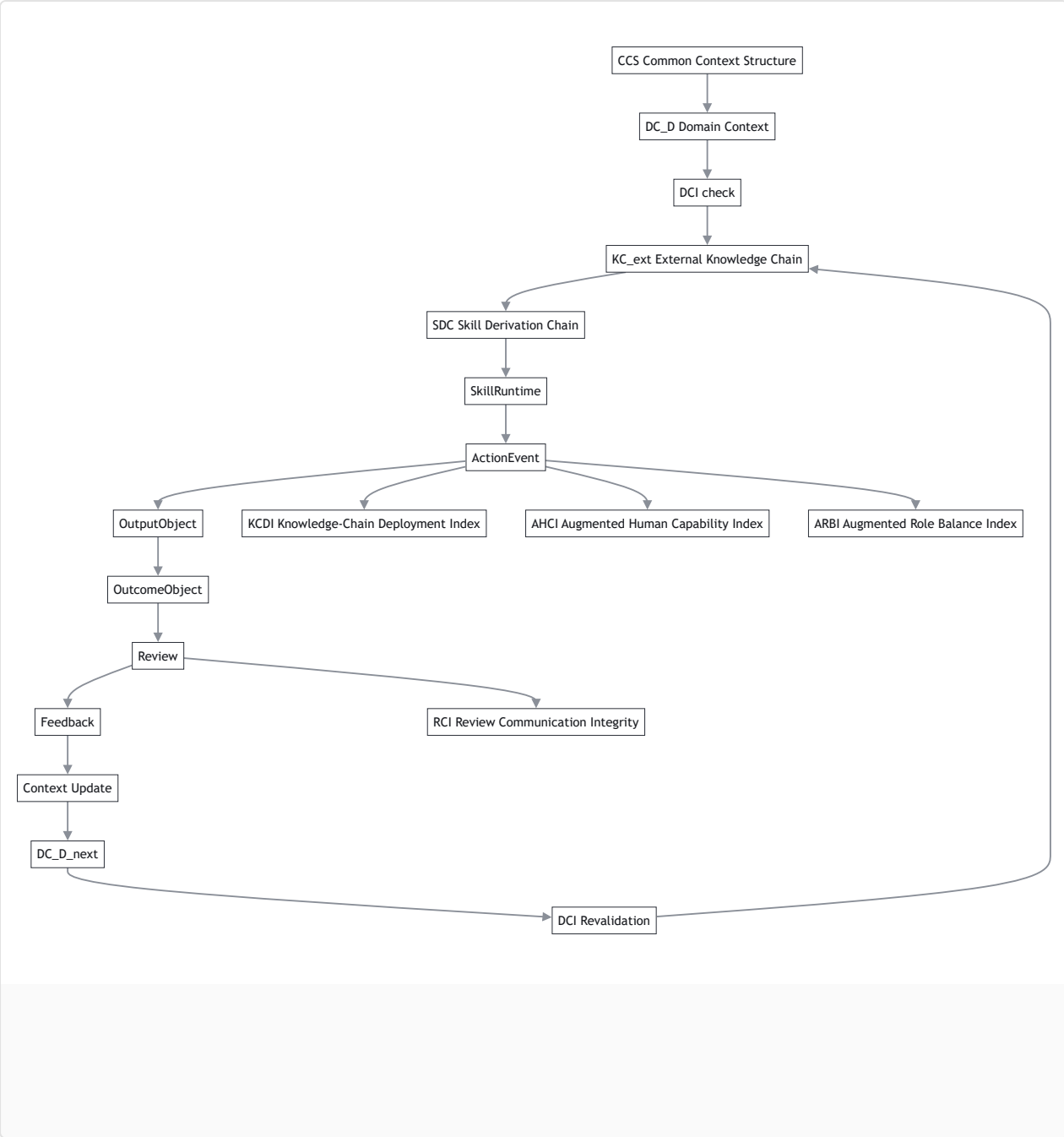


Figure 4. KAC with operational metrics — KCDI, AHCI, and ARBI branch from ActionEvent, and RCI from Review, to measure execution and observation.

11 The Integrated Cycle of KAC

KAC is not a linear process but a cyclic structure**[R6]** (*Figure 2*).

CCS -> DC_D -> DCI -> Identity-based knowledge chain -> ValidDerivation -> ValidDeployment
-> SkillRuntime -> ActionEvent -> OutputObject -> OutcomeObject
-> KCDI / AHCI / ARBI / RCI -> Feedback -> DC_D(t+1) -> DCI Revalidation

Figure 5. The integrated cycle of KAC — DCI verifies the upstream reference environment, the knowledge chain derives the Skill, SkillRuntime and ActionEvent record execution, the four metrics measure the result, and approved Feedback creates a new version that closes back into DCI verification.

When this cycle closes, the organization has not merely used AI but accumulated AI execution results as organizational knowledge and criteria.

12 Example: Customer-Service KAC

Applying KAC to the customer-service domain yields *Table 6*.

Table 6. KAC applied to the customer-service domain

KAC element	Customer-service example
KC_ext	Product policy, refund rules, customer history, legal criteria, FAQ
SDC	CustomerSupportAgent → CustomerIssueResolution → IssueClassification → ProductPolicyKnowledge → ResponseFramingMethod → Policy-grounded Response Skill
SkillRuntime	Input: customer inquiry / Sources: FAQ, terms / Guardrail: no legal advice / Approval: human approval for refund and compensation cases
ActionEvent	AI classifies the inquiry and generates a response draft
OutputObject	A customer response draft with linked grounds
OutcomeObject	Inquiry resolved, re-inquiries reduced, no policy violation
Review	RCI reviews grounds, authority, accountability, expression, and approval
Feedback	Registers "refund exception criteria are ambiguous" as a change candidate
DC_D(t+1)	Updates the refund criteria in the customer-service domain context

What matters here is not the fact that AI produced a response draft, but from which knowledge-chain-derived Skill it was based, through which Runtime it was executed, how the execution event was recorded, how output and outcome were separated, and how review and feedback improved the domain context. That is — **AI response generation is only part of KAC; the essence of KAC is that knowledge is verified through action and that the result of that action improves organizational criteria.**

13 The Significance of KAC

- **It fills the gap between knowledge and action.** Many organizations build knowledge stores, manuals, prompts, RAG, and agents separately, yet cannot explain into which Skill knowledge is derived or through which execution contract it becomes action. KAC connects this as *Knowledge → Skill → Runtime → Action → Outcome*.
- **It redefines Skill as the AX execution unit.** Real work requires source exploration, criteria application, judgment, verification, expression, approval, recording, and improvement, so the execution unit is not raw data but a Skill**[R7]**; KAC addresses the derivation, execution, and verification of that Skill.
- **It distinguishes output from outcome.** A document being produced does not mean knowledge was accumulated. The distinction between OutputObject (artifact) and OutcomeObject (change) enables evaluating not "what AI produced" but "what change it

produced."

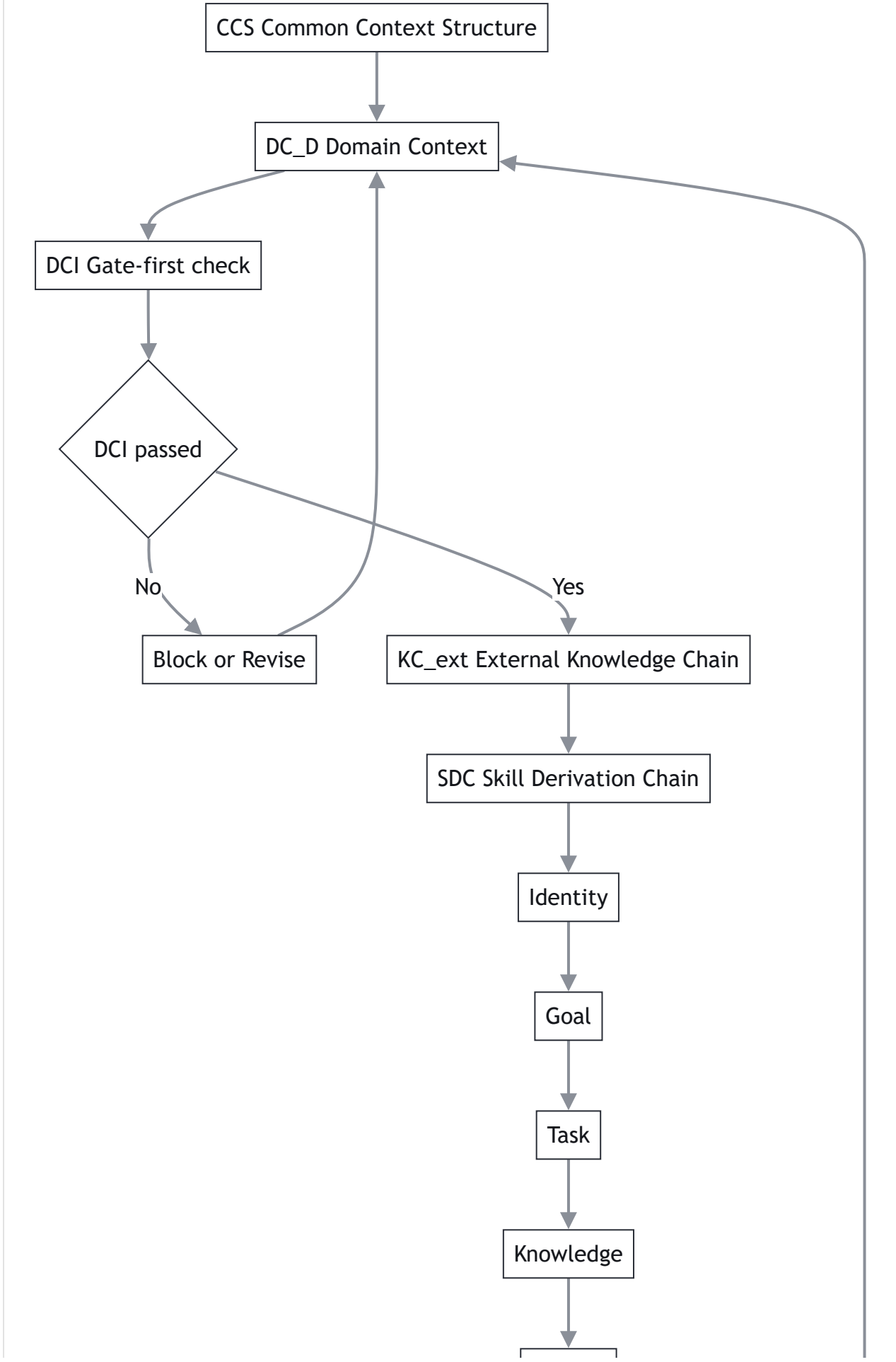
- **It includes Review and Feedback in the execution chain.** AI results must not be reflected into organizational criteria without review; RCI verifies utterances and artifacts as review objects and lets only approved feedback become context change candidates**[R3]**.
- **It makes AX an organizational learning structure.** Without KAC, AI use scatters into individual artifact production; with KAC, execution results are recorded, reviewed, fed back, and accumulated into a new version of the domain context.

14 Conclusion

Organizational competitiveness in the age of AI is not determined by which AI model is used alone. More important are: within which common and domain context the model operates, through which knowledge chain it derives a Skill, through which SkillRuntime that Skill is executed, and how execution results are verified and learned into organizational criteria. A knowledge chain is a path of knowledge, and a skill derivation chain is a reasoning path explaining why a given Identity or Entity requires a given Skill. But organizational AX cannot stop there — the derived Skill must be compiled into a Runtime, executed as an ActionEvent, leave an OutputObject and OutcomeObject, and pass Review to be organized into Feedback that updates the domain context. This entire structure is KAC.

> Without KAC, knowledge is stored.
> With KAC, knowledge acts. (13)

More precisely — **KAC is the AX execution–verification–learning ontology along which validated knowledge is derived into an executable Skill, invoked as an ActionEvent through a SkillRuntime, producing Output and Outcome, and updating the domain context through Review and Feedback.** KAC is therefore not a mere knowledge-management model but an execution operating system by which an organization in the age of AI turns knowledge into action, verifies action through outcome, and accumulates outcome back into the organization's criteria and context.


CCS Common Context Structure
DC_D Domain Context
DCI Gate-first check
DCI passed
No
Yes
Block or Revise
KC_ext External Knowledge Chain
SDC Skill Derivation Chain
Identity
Goal
Task
Knowledge

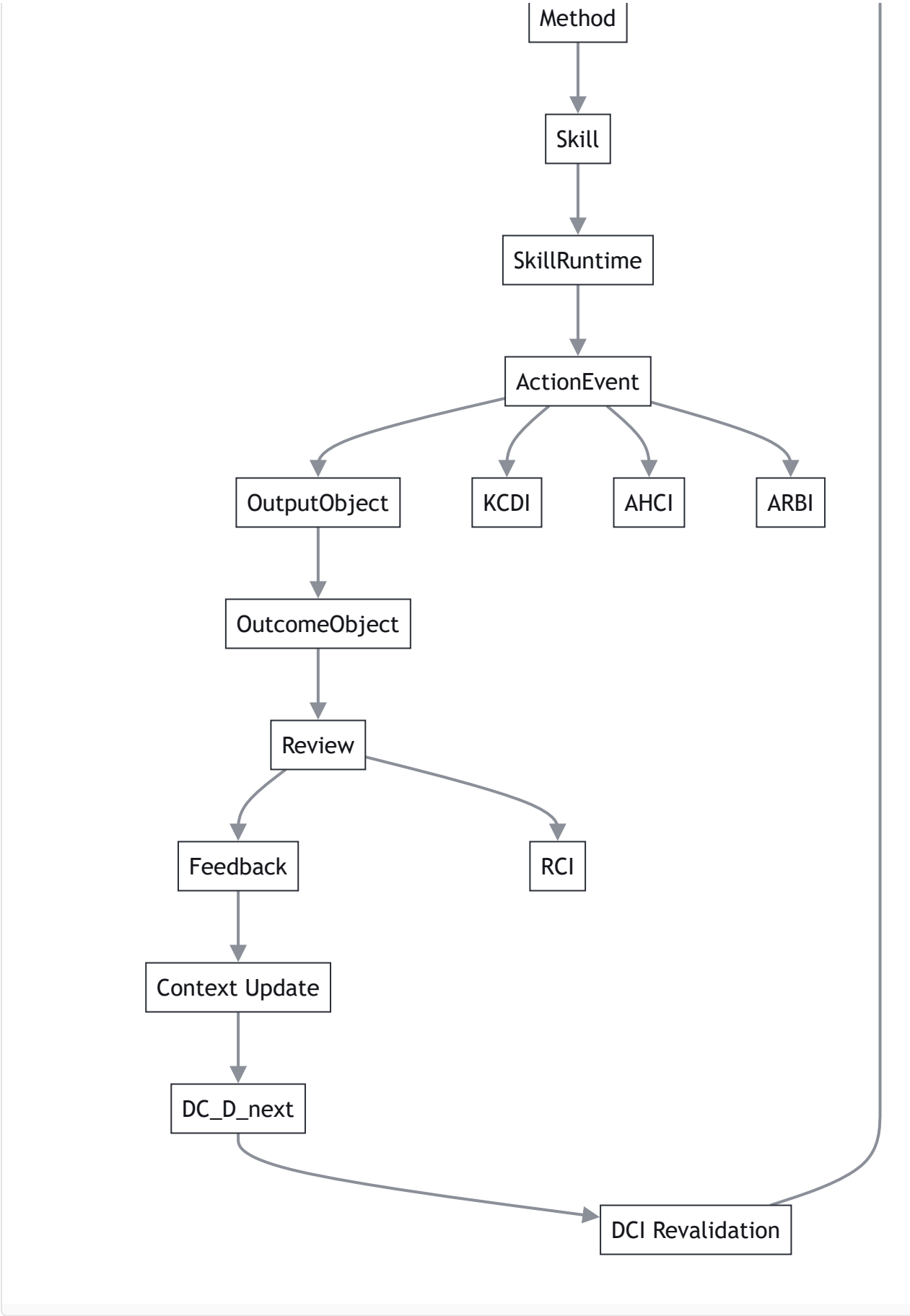


Figure 6. Full integrated diagram — DCI Gate-first decision (pass / Block or Revise), the SDC expansion, the execution chain, and the KCDI / AHCI / ARBI / RCI metric branches unified into one structure.

Author contribution & sources. The KAC nine-tuple, SkillRuntime, the separation of ActionEvent / OutputObject / OutcomeObject, the ValidKAC condition, the linkage with DCI / KCDI / RCI / AHCI / ARBI, and the integrated cycle in this paper were organized and systematized from the manuscript "Knowledge–Action Chain (KAC) final draft v1.0" and organizational-AX research materials on the knowledge chain, KCDI, RCI, DCI, ARBI, AHCI, common/governance context, and the integrated AX cycle. Descriptions of the external knowledge chain, Knowledge Value Chain, and Knowledge-to-Action rest on the external references below. KAC is an applied execution ontology proposed by this framework; the verification criteria and coefficients of each segment are operational parameters to be calibrated with real domain data.

References

[R1] Organizational AX research, "Development and Deployment of an Identity-driven Skill Derivation Knowledge Chain," unpublished manuscript, 2026.

[R2] Organizational AX research, "Knowledge-Chain Deployment Index (KCDI)," unpublished manuscript, 2026.

[R3] Organizational AX research, "Review Communication Integrity (RCI v3.0)," unpublished manuscript, 2026.

[R4] Organizational AX research, "Common Context and Governance Context," unpublished manuscript, 2026.

[R5] Organizational AX research, "Augmented Role Balance Index (ARBI)" and "Augmented Human Capability Index (AHCI)," unpublished manuscripts, 2026.

[R6] Organizational AX research, "A Common-Context-Based Integrated Cycle of AX Execution, Verification, and Learning" and "Domain Context Integrity (DCI)," unpublished manuscripts, 2026.

[R7] C. W. Holsapple, M. Singh, "The Knowledge Chain Model: Activities for Competitiveness," *in Handbook on Knowledge Management, Springer.*

[R8] IAEA, "Knowledge Management — Knowledge Value Chain." *https://gnssn.iaea.org/main/Pages/Capacity Building/Knowledge_Management.aspx*

[R9] Registered Nurses' Association of Ontario (RNAO), "Knowledge-to-Action Framework." *https://rnao.ca/bpg/leading-change-toolkit/knowledge-to-action-framework*